A Safety Profile Verifying Class Initialization

Document #:	P3402R0
Date:	2024-09-17
Project:	Programming Language C++
Audience:	SG23

Reply-to:	Marc-André Laverdière, Black Duck Software <marc-andre.laverdiere@blackduck.com> Christopher Lapkowski, Black Duck Software <redacted@blackduck.com> Charles-Henri Gros, Black Duck Software <redacted@blackduck.com>

Contents

§ 1 Abstract

We propose an attribute that specifies that every data member that belongs to a verified class has all its data members initialized to determinate values, assuming that the data used for construction is itself initialized properly. Thus, when the assumptions are validated, none of its data members can cause undefined or erroneous behavior.

This safety profile restricts what kind of code a constructor can have. Existing code bases are likely to violate these constraints, and thus this feature is an opt-in.

§ 2 Introduction

There is a growing push towards memory-safe languages. While C++ is not memory-safe, it is desirable to specify and opt-in mechanism allowing a subset of C++ features that would result in memory safe-languages. This has been termed 'profiles'([P3274R0]), and would be specified at the TU level using an attribute. In this paper, we propose `[[Profiles::enable(initialization)]]`.

All classes under the purview of the profile attribute will have the guarantee that all its data members are properly initialized once the object is constructed, assuming that the data used for construction is itself initialized properly. In the case of a class that inherits from one or more classes, all its base classes must be compliant with this profile.

Example:

```
struct [[Profiles::enable(initialization)]] parent1 {
  int i;
  parent1() = default; //non-compliant, i is default initialized
};
struct [[Profiles::enable(initialization)]] child1 : public parent {
  int j;
  child1() : parent1(), j(42) {} //child is compliant, but parent isn't
}

//Not a verified class, but would be compliant if it were
struct parent2 {
  int i = 0;
  parent2() = default;
};
struct [[Profiles::enable(initialization)]] child2 : public parent2 {
  int j;
  child() : parent2(), j(42) {} //child is not compliant, because parent2
        is not a verified class
}
```

§ 3 Definitions

A *verified class* is a class that is affected by the profile annotation, or a scalar type.

An *object parameter* is either `this` or an explicit object parameter.

A *verified data member* is a data member that is not exempted from verification.

§ 4 Location of the Profile Annotation

In this paper, we give examples with the profile annotation attached to specific classes. This deviates from [P3274R0], which suggested annotations at the translation unit level. We do so to make it clear which classes are verified classes and which ones aren't, since some examples have a mix of them. Our proposal is orthogonal to the location of the annotation.

§ 5 Exemptions

Some data members are exempt from verification, either due to intrinsic properties, or due to explicit opt-out from the developer.

§ 5.1 Constantly-Initialized Static Data Members

The presence of static data members ([class.static.data]) in a class are allowed in this profile, but the profile offers minimal guarantees.

Static data members have either static storage duration ([basic.stc.static]) or thread storage duration ([basic.stc.thread]). They are guaranteed to be initialized with constant initialization ([basic.start.static]). However, they can be reassigned during dynamic initialization ([basic.start.dynamic]).

Dynamic initialization can lead to subtle bugs, such as:

— Circular dependencies

— Use of statically-initialized values before they are dynamically-initialized.

— Initialization order issues when dealing with symbols variables defined in other translation units.

— Assignment of uninitialized values

All static data members belonging to verified classes that are initialized solely using constant initialization are exempted from verification. All other static data members are non-compliant with this profile.

```
int randomInt() {
    int therandomint;
    return therandomint;
}
struct [[Profiles::enable(initialization)]] WithStaticUninit1 {
    WithStaticUninit1() = default; //Not a POD
    static int thestatic;
};
int WithStaticUninit1::thestatic = randomInt(); //non-compliant
```

```
struct [[Profiles::enable(initialization)]] GetsCorrupted {
    GetsCorrupted() : thefield(0) {} //compliant
    int thefield;
};

struct [[Profiles::enable(initialization)]] Wrapper {
    Wrapper() = default; //Not a POD
    static GetsCorrupted wrapped;
};
GetsCorrupted corruptingFactory() {
    GetsCorrupted ret{};  //All initialized, good
    ret.thefield = randomInt(); //Now, some uninitialized memory snuck in
    return ret;
}
GetsCorrupted Wrapper::wrapped = corruptingFactory(); //Non-compliant
```

§ 5.2 Explicit Exemptions

[P3274R0] mentions that performance critical applications won't initialize output buffers at first and mentions a few possibilities: "suppression, an uninitialized annotation, and/or by specific uninitialized types."

This profile allows a pointer to be initialized to any value, whether that's a `nullptr`, the return value of a `new`, or even an hardcoded address. It does not require that the memory space pointed to by the pointer is set to any value. This is an allowance for systems programming, which sometimes have buffers pointing to hardcoded addresses, which are used for interacting with devices. We leave the question of pointer safety to `[[Profiles::enable(Pointers)]]`.

§ 5.2.1 Exemption by Annotation

Developers who require to exempt specific data members from verification may use the `[[indeterminate]]` annotation from [P2795R5].

§ 5.2.2 Exemption by Type

A better solution is to use a specialized type offers restricted access to memory regions. We envision a class named `std::RawBuffer<T>`, which would record which regions of the buffer have been written previously, and prohibit reads outside of that region. This class would have use beyond this profile, making it a more generic solution.

§ 6 Verification of Constructors

All constructors of a verified class must satisfy the following properties:

— All base classes must be verified classes.

— All base classes must be initialized before any of their non-static data members are read.

— All verified data members must be initialized before being read.

— All verified data members must be initialized on all paths.

— No data member that is explicitly exempted, or any of its data members, can be read.

— Any use of an object parameter is restricted until all non-static data members are initialized. When restricted, the object parameter can only be used for accessing non-static data members.

— The constructor may only call constructors of verified classes until all non-static data members are initialized. Calls to other functions are prohibited until then.

— The return value of function calls may never be assigned to an data member, either directly or indirectly.

— Default initialization is not considered as initializing anything for the sake of these criteria.

A non-static data member is considered read whenever it is present in the function, except when:

— It is an unevaluated operand [expr.context]

— It is part of a discarded statement [stmt.if]

— It is the left operand of an assignment [expr.ass]

— It is the receiver object of a constructor invocation

This definition implies the following:

— Obtaining the address of a data members violates this profile.

— A verified class' non-static data members are either verified classes or PODs. Note that PODs don't have constructors to call, even though the syntax used may give that impression.

— A nonconforming constructor would bring the rejection of its class only, and not of its subclasses. This decision is intended to reduce the noisiness that would come from a faulty constructor at the top of a very large class hierarchy.

— An initialized pointer can point to an uninitialized memory region (e.g. `buf(new char[BUF_SIZE]))`) and be compliant. We discuss this above.

— The constructor may execute nearly arbritary code after its non-static data members are intitialized.

We restrict function calls after initialization because we want to keep the analysis intraprocedural. Initialization that occurs in a member function, or occurs from a function's return value, would require either interprocedural analysis. Keeping the analysis intraprocedural would faciliate adoption.

Examples:

```
struct [[Profiles::enable(initialization)]] clazz1 {
  int i;
  int j;
  int z = 0;
  clazz1() {
      i = 123;
      if (nondet) {
        j = 456;
      }
      //non-compliant: j is not defined on all paths
  }
};
```

```
struct [[Profiles::enable(initialization)]] clazz2 {
  int i;
  int j;
   clazz2() : i(j), j(42) {} //non-compliant: j is read before it is
        initialized
};
```

```
struct [[Profiles::enable(initialization)]] clazz3 {
  int i;
  int j;
  clazz3() { //compliant, but bad form
    this->i = 0;
    this->j = 42;
  }
};
```

```
struct pod {
  int i;
  int j;
};
struct [[Profiles::enable(initialization)]] clazz4 {
  pod p;
  clazz4() = default; //non-compliant, p is default initialized
};
struct [[Profiles::enable(initialization)]] clazz5 {
  pod p{};
  clazz5() = default; //compliant, p is value-initialized
}

struct [[Profiles::enable(initialization)]] clazz6 {
  pod podFactory() {
    pod p;    //p is default-initialized
    return p;
  }
   clazz6() : p(podFactory()) {} ; //non-compliant, p initialized from a
        function's return value
}
```

```
struct [[Profiles::enable(initialization)]] clazz7 {
  int i;
  int j;
```

```
  clazz7(int i) : i(i), j() {}; //compliant, j is value-initialized
};
```

§ 7 Templates

In the case of templated classes, the property is verified during template instantiation.

```
class NotAnnotated{/**/};
class [[Profiles::enable(initialization)]] Annotated {/**/};

template<typename T>
class [[Profiles::enable(initialization)]] AnnotatedTemplate {
  T field = T();
};

void foo() {
   AnnotatedTemplate<NotAnnotated> nat {}; //non-compliant, calling the
        constructor to a non-verified class
  AnnotatedTemplate<Annotated>     at {}; //compliant
}
```

§ 8 Idioms to Consider

§ 8.1 Non-Constructor Delegating

A recent SG23 ML discussion highlighted that it is idiomatic in C++ to delegate initialization to a non-constructor method. Supporting this idiom would make this profile more useful.

The bit of code that triggered the discussion is the following:

```
basic_string(const _CharT* __s, const _Alloc& __a = _Alloc())
: _M_dataplus(_M_local_data(), __a)
{
  //...
  _M_construct(__s, __end, forward_iterator_tag());
}
```

In this case, `_M_local_data()` returns a const pointer to a data member (`_M_local_buf`) and passes it to the `_M_dataplus` data member. The initialization then is done by `_M_construct`. This code would be reported as violating the safety profile as we specify it in this draft, since the constructor does not initialize `_M_dataplus` directly.

There are a few solutions to this problem:

1. Force developers to rewrite their code to use delegating constructors.
2. Use an on-demand interprocedural analysis that ensures that initialization happens on all paths in the callee.

3. A suggestion was to annotate arguments that must be initialized with `[[must_init]]`. This option is less intrusive than option 1, but adding another annotation is undesirable. It nonetheless would be simpler to verify than option 2, simply because the scope of the analysis becomes well-bounded. As such, it is worth considering.

It would be a good idea to have a straw poll about which of these options the community considers best suited.

§ 9 Future Work

The proposed attribute could be misunderstood to mean that all variables in all the code in the scope of the annotation are properly initialized. This may be addressed in a future version of this profile, or in another profile.

We also observe that profiles imply a constraint on what types can be used in a template. This hints at a new concept. A future revision of this paper would explore this further.

§ 10 Conclusion

In this paper, we propose a safety profile that guarantees that any class affected by the profile annotation will have all its data members initialized, assuming that the data used for construction is itself initialized properly. The profile does not depend on the presence of specific modern C++ features and can thus be applied to legacy code bases.

§ 11 References

[P2795R5] Thomas Köppe. 2024-03-22. Erroneous behaviour for uninitialized reads. https://wg21.link/p2795r5

[P3274R0] Bjarne Stroustrup. 2024-05-10. A framework for Profiles development. https://wg21.link/p3274r0